

Сначала *план*. Потом действие.

Маленькие задачи делаются с первого промпта. Крупные — через Plan Mode, спецификации и дисциплину чек-поинтов. На этом этапе анализируем годовые выписки и считаем, когда вы выйдете на финансовую независимость — в трёх сценариях.

ДЛИТЕЛЬНОСТЬ

5–7 дней · 2–4 ч/день

ЧТО ВЫ ОСВОИТЕ

Plan Mode · TodoWrite · spec.md · /compact · verification

ГЛАВНЫЙ ИТОГ

крупные задачи без потери цели

АРТЕФАКТ

отчёт «куда ушли деньги» и прогноз финансовой независимости в трёх сценариях

Главные инструменты этапа – Plan Mode, TodoWrite и `/compact` – уже встроены в Claude Code, ставить их не нужно. Опционально доустановим два skill из набора `superpowers` для дисциплины планов.

ОБЯЗАТЕЛЬНО

Из этапов 00–03

- Claude Code, MCP `@playwright/mcp` и `@upstash/context7-mcp` – ставили в этапе 03.
- Skill `frontend-design` – для графиков прогноза.
- Привычка работать через `CLAUDE.md` и итерации.

ЖЕЛАТЕЛЬНО • SKILL

Skill `superpowers:writing-plans` и `superpowers:executing-plans`

Что это: два skill из пакета `superpowers`. Первый учит Claude составлять структурированный план перед крупной задачей; второй – исполнять план поэтапно, с чек-поинтами и остановками для вашего ревью.

Зачем сейчас: для второго проекта (прогноз с тремя сценариями) полезно работать по дисциплине: написать spec → план → исполнение по плану. Без skills тоже можно, но со skills меньше «уплываний».

Проверить, не стоят ли уже: в Claude напечатайте `/skills`. Если вы ставили пакет в этапе 01, `superpowers:writing-plans` и `superpowers:executing-plans` уже в списке. Если нет – команда ниже поставит сразу весь пакет.

```
# внутри claude (slash-команда, не shell)
/plugin install superpowers@claude-plugins-official
```

Где взять: встроенный маркетплейс Claude Code (`claude-plugins-official`); исходники – github.com/obra/superpowers. Один пакет, внутри – набор связанных skills (planning, brainstorming, debugging, TDD, verification).

Заметка: в отличие от MCP и skills из этапа 03, этот пакет живёт не в Smithery, а во встроенном маркетплейсе Claude Code. Поэтому команда другая – slash-команда внутри `claude`, не shell. Магазинов несколько, это нормально.

Проверка: попросите Claude «напиши spec и план для трёх-сценарного прогноза сбережений» – должен составить именно spec + план, а не броситься писать код.

Skill superpowers:verification-before-completion

Что это: skill, который учит Claude *не объявлять задачу выполненной*, пока он не запустил проверку: открыл результат в браузере, проверил числа на тестовом сценарии, сверил с критериями готовности.

Зачем сейчас: крупные задачи – идеальное место «соврать про готовность». Skill это блокирует.

Где взять: входит в тот же пакет `superpowers`, ставится той же командой выше.

ДАННЫЕ

Банковские выписки за 12 месяцев (для проекта А)

Что это: CSV или PDF выписки от вашего банка за последние 12 месяцев.

Зачем сейчас: на них строим отчёт «куда ушли деньги».

Где взять: в кабинете банка обычно есть «выгрузить выписку за период». Если своих нет – в первом промпте попросим Claude сгенерировать тестовые на ~600 операций.

Куда положить: `data/statements/` в корне проекта. Никогда не коммитить в git.

Зачем нужен план перед действием

На этапах 00–03 мы работали короткими шагами: один промпт — одна правка — видим результат. Этого хватает на задачи в ~5–10 шагов.

Но есть задачи на 30–50 шагов: «проанализировать 12 банковских выписок, категоризировать ~600 операций, сделать отчёт с графиками и тремя инсайтами». Если делать «одним промптом» — Claude быстро «уплывает»: забывает критерии, теряет цель, оставляет половину работы.

Решение — **двухфазная работа**:

1. **Фаза «продумай»** — никаких изменений в файлах. Claude составляет план, перечисляет шаги, оценивает риски, спрашивает уточнения.
2. **Фаза «сделай»** — вы утверждаете план, Claude его исполняет, отмечая прогресс.

Эта философия пронизывает весь этап. Plan Mode, spec.md, TodoWrite, verification — все они служат одному: *сначала договориться о том, что будет сделано, потом делать*.

Plan Mode подробно: как и когда

Plan Mode — это специальный режим Claude Code, в котором он *гарантированно* не трогает ваши файлы, пока вы не утвердите план. Включается через `Shift+Tab` (см. этап 00).

Что Claude делает в Plan Mode:

- Читает существующие файлы и `CLAUDE.md` (это разрешено);
- Изучает зависимости, структуру, ваши данные;
- Составляет план: фазы, шаги, риски, вопросы;
- Останавливается, ждёт вашего «да / нет / поправь X».

После «да» план попадает в обычный режим, и Claude его выполняет. Если план слишком большой — Claude сам предложит разбить на несколько подзадач.

Когда обязательно использовать Plan Mode:

- задача займёт больше 10 шагов;
- задача завязана на ваши данные и ошибка дорого стоит (бюджет, инвестиции, налоги);
- вы не уверены, что Claude правильно понял требования;
- задача затрагивает много файлов одновременно.

Когда не нужно: простые правки, мелкие итерации – они тратят время на «пустой план».

TodoWrite: видимый прогресс по шагам

`TodoWrite` – встроенный инструмент Claude Code. Когда задача состоит из 3+ шагов, Claude автоматически создаёт список задач и показывает его в диалоге. Каждая задача имеет статус:

- `pending` – ещё не начата;
- `in_progress` – делается прямо сейчас (только одна за раз);
- `completed` – готова.

Зачем это вам:

1. **Видите, где Claude.** «Сейчас в шаге 3 из 7» – не нужно гадать.
2. **Видите, что забыто.** Если в конце все шаги `completed`, кроме одного `pending` – это явный сигнал.
3. **Можете остановить и вернуться.** Сессия запомнит todo-лист – через `--continue` вы возвращаетесь к тому же месту.

Можно явно попросить Claude использовать TodoWrite:

```
> разбей задачу «анализ выписок» на 5–8 фаз
> через TodoWrite. потом приступай.
```

Спецификация (spec.md): техзадание для Claude

Когда задача очень большая (десятки шагов, недели работы), даже Plan Mode мало. В таких случаях работает **spec-driven development**: вы вместе с Claude пишете `spec.md` – документ, который описывает задачу *до начала работы*.

Что в spec обязательно:

1. **Цель.** «Что мы строим» в одном абзаце.

2. **Входы.** Какие данные на входе. Где они лежат, в каком формате.
3. **Выходы.** Что должно появиться. Файлы, страницы, отчёты, цифры.
4. **Шаги.** Высокоуровневые фазы работы (5–10 пунктов).
5. **Критерии готовности.** «Считаем готовым, когда...» — список измеримых условий.
6. **Что вне score.** Что мы *не будем* делать (важно для Claude, чтобы он не уплывал).
7. **Риски и вопросы.** Открытые места, где нужно ваше решение.

Спец живёт в `specs/` или прямо в корне проекта — рядом с `CLAUDE.md`.
На него ссылаются через `@spec.md`:

```
> работаем по @spec.md.  
> начни с фазы 1, через TodoWrite води прогресс.  
> останавливайся в конце каждой фазы — я буду ревьюить.
```

Compaction и управление длинными сессиями

В долгой сессии (часы работы, десятки промптов) у Claude накапливается «контекстный мусор» — старые ответы, отменённые попытки, тестовые запросы. Это *замедляет* Claude (больше текста на обработку) и *сбивает*: он может вспомнить отменённое решение и применить заново.

Команда `/compact` — «уплотни сессию»:

1. Claude суммаризирует всё, что было: цель, ключевые решения, текущее состояние;
2. Удаляет промежуточные сообщения;
3. Оставляет суммарный «отчёт о текущем положении» как первое сообщение в новом, чистом контексте.

Когда уплотнять:

- после каждой большой фазы плана (например, «парсинг готов — `/compact`, дальше категоризация»);
- когда Claude начал «забывать» решения, принятые часом ранее;
- в конце рабочего дня — чтобы завтра `--continue` начался с чистого, но информированного контекста.

Альтернативы:

- `/clear` – полная очистка. Контекст теряется. Полезно, когда хотите начать «с нуля» в той же папке.
- `--resume` – выбрать одну из недавних сессий, если случайно открыли новую.

Verification: «готово» только после проверки

Самая частая ошибка крупных задач – «соврать про готовность». Claude может написать код, увидеть, что он *выглядит* правильно, и объявить задачу законченной. Запустите – не работает.

Дисциплина **verification-before-completion** требует прежде чем сказать «ГОТОВО»:

1. **Запустить** – программу, страницу, расчёт.
2. **Прогнать тестовый сценарий** – известный вход, ожидаемый выход.
3. **Сверить с критериями готовности** из `spec.md`.
4. **Через Playwright** сделать скриншот, посмотреть на UX.
5. **Только потом** – объявить задачу выполненной.

Skill `superpowers:verification-before-completion` встраивает эту дисциплину в поведение Claude. Без skill можно делать ручную:

```
> прежде чем сказать «готово», прогони такой тест:  
> на входе X – должно получиться Y. сверь.  
> через playwright открой страницу, сделай скриншот.  
> покажи мне результат каждой проверки.
```

Декомпозиция: когда и как разбивать задачу

Признаки, что задача слишком большая для одного промпта:

- больше 10 шагов;
- требует нескольких источников данных;
- содержит несколько независимых выходов (отчёт + дашборд + автоматизация);
- вы сами не уверены, что должен сделать Claude.

Как разбивать:

1. **На фазы по «данным».** Парсинг → нормализация → агрегация → визуализация → отчёт.
2. **На фазы по «артефактам».** Сначала `data/operations.csv`, потом `dashboard.html`, потом `report.md`.
3. **На фазы по «ревью».** Каждая фаза заканчивается `/compact` и вашим утверждением.

Каждая фаза должна быть:

- самостоятельно полезной (можно остановиться и использовать промежуточный результат);
 - измеримой («готово, когда X существует и показывает Y»);
 - укладываться в 1–3 часа.
-

Спираль: спес → план → исполнение → ревизия

PRACTICUM

Прогон • Маленький спес на настоящей задаче

1. Возвращаемся в `portfolio-journal` из этапа 02. Задача: добавить страницу `monthly.html` с помесечной разбивкой по тикерам.
2. Войдите в Plan Mode (`Shift+Tab` до «plan»). Промпт:

```
> составь спес.md для новой страницы monthly.html.  
> источник данных: @data/operations.csv.  
> цель: помесечная разбивка операций по тикерам с трендами.  
> в спес включи цель, входы, выходы, шаги, критерии готовности,  
> что вне scope, открытые вопросы.
```

3. Прочитайте получившийся `спес.md`. Поправьте, если что-то не так. Сохраните.
4. Скажите Claude утвердить план: *«план одобрен, исполняй фаза за фазой через TodoWrite»*. Выйдите из Plan Mode (`Shift+Tab` до «default»).
5. Наблюдайте, как Claude отмечает прогресс. После каждой фазы: «open monthly.html, screenshot via playwright, что вижу – рассказывай».
6. В конце – verification:

```
> прежде чем сказать «готово», прогони сверку:  
> – все ли месяцы из CSV отображаются?  
> – итоги по тикерам совпадают с dashboard.html?  
> – на узком окне (телефон) ничего не ломается?  
> покажи результат каждой проверки.
```

7. После завершения – `/compact`. Сессия станет лёгкой – следующая задача начнётся с чистого, но информированного контекста.

Что мы тренируем: **четырёхшаговый цикл – спес, план, исполнение, verification**.
Дальше его применяем на реальных проектах.

ПРОЕКТ А · ПО ИНСТРУКЦИИ

«Куда ушли деньги за год»

Цель: 12 банковских выписок → парсинг → категоризация → отчёт `report.md` с графиками и тремя ключевыми инсайтами. Дисциплина: *spec* → *Plan Mode* → *TodoWrite* → *verification*.

Шаги

1. Создайте папку `year-review`. Положите выписки в `data/statements/` (свои или попросите Claude сгенерировать ~600 операций).
2. Запустите Claude. Войдите в Plan Mode. Промпт:

```
> составь spec.md для проекта «куда ушли деньги за год».  
> источник: @data/statements/ (12 csv-файлов, по одному на месяц).  
> цель: report.md с пятью разделами:  
>   - total spend / income за год;  
>   - топ-10 категорий с долями;  
>   - три «удивительных» инсайта (что бросилось в глаза);  
>   - тренд «доход – расход» по месяцам;  
>   - три «куда смотреть в следующем году».  
> формат: markdown + графики (chart.js, через context7).  
> критерии готовности – измеримые, перечисли.
```

3. Утвердите `spec.md`. Скажите: «работаем по @spec.md, через TodoWrite, фазами с ревью между ними».

4. Выйдите в default. Пройдите фазы (Claude сам разобьёт):

- a. **Парсинг:** все 12 файлов → один `data/all-ops.csv` с нормализованными колонками.
- b. **Категоризация:** каждой операции – категория. Используйте «правила» (ключевые слова в описании) + ручную проверку.
- c. **Агрегация:** суммы по категориям, по месяцам.
- d. **Визуализация:** круговая диаграмма категорий, столбчатая по месяцам, тренд «доход – расход».
- e. **Инсайты:** Claude находит три неожиданных факта в данных.
- f. **Финал:** сборка `report.md`.

5. После каждой фазы – `/compact` и ваше «давай дальше» или «поправь X».

6. Verification в конце:

```
> прогони sanity-check перед «готово»:  
> – total operations в report.md == строка в data/all-ops.csv;  
> – total spend == сумма всех «расход»-строк;  
> – суммы категорий складываются в total;  
> – все 12 месяцев присутствуют в графике тренда.  
> покажи каждый чек как «✓» или «X» + что не сходится.
```

7. Откройте `report.md` в браузере (markdown viewer) или превратите в HTML через Claude. Прочитайте – должны узнать о себе что-то новое.

Что вы здесь освоили

- полный цикл spec → план → фазное исполнение → verification;
- работу с большим объёмом данных (~600 операций) без «уплываний»;
- дисциплину чек-поинтов и `/compact` между фазами;
- выпуск читаемого аналитического отчёта в markdown.

Прогноз финансовой независимости в трёх сценариях

Цель: интерактивная страница, которая отвечает на вопрос «когда я смогу не работать ради денег», в трёх сценариях: консервативный, реалистичный, агрессивный. Графики и «критическая дата» для каждого.

Шаги

1. Папка `fi-forecast`. Запустите Claude в Plan Mode.

2. Спец:

```
> составь spec.md для «прогноза финансовой независимости».
> входы (поля формы):
>   – текущий капитал, ежемесячный доход, расход,
>   – горизонт (лет), целевой пассивный доход (₽/мес),
>   – три набора параметров (см. ниже).
> три сценария:
>   conservative: доходность 5%, инфляция 6%, рост дохода 0%
>   realistic:     доходность 8%, инфляция 5%, рост дохода 3%
>   aggressive:   доходность 12%, инфляция 4%, рост дохода 7%
> выходы:
>   – таблица «капитал по годам» для каждого сценария;
>   – общий график (три линии);
>   – три «критические даты» — год, когда пассивный доход
>     перекрывает целевой;
>   – текстовое summary в трёх предложениях.
> критерии готовности — измеримые.
```

3. Утвердите spec. Дайте команду исполнять.

4. Фазы (Claude разобьёт):

- a. Скелет страницы (форма + пустые блоки результата);
- b. Чистая логика расчёта (сначала консольно, проверить на тестовых вводах);
- c. Подключить расчёт к форме;
- d. Графики (chart.js через context7);
- e. Интерпретация: «критические даты» и три-предложение саммари;
- f. UX-полировка через `frontend-design` + Playwright-ревизия.

5. Между фазами — `/compact` и ваше ревью.

6. Verification обязательна:

```
> sanity-check:  
> – на тестовом вводе (капитал 1М, доход 100к, расход 70к,  
> горизонт 30 лет, цель 50к/мес) реалистичный сценарий  
> должен показывать критическую дату через 12–14 лет.  
> – у консервативного дата позже, у агрессивного раньше.  
> – графики стартуют с одной точки и расходятся.  
> – форма работает на узком экране.  
> покажи результаты каждой проверки.
```

7. Финальный аудит-вопрос: *«какие три самых хрупких допущения в этой модели? что произойдёт, если инфляция в реальности будет в два раза выше?»*

Что вы здесь освоили

- спес на многосценарную задачу с интерактивностью;
- тестирование расчётной логики на известных входах перед UI;
- верификация и через числа, и через UX (Playwright-скриншот);
- аудит «хрупких допущений» модели.

ПРОЕКТ С · САМОСТОЯТЕЛЬНО

Симулятор пенсии в четырёх стратегиях

Цель: вариация проекта В – страница, где вы сравниваете четыре стратегии накопления (ИИС / ETF / валюта / недвижимость) на 30-летнем горизонте. Те же шаги: *spec* → план → фазы → *verification*.

Подсказки

- Структура та же – форма с общими параметрами (доход, отчисление в месяц), результаты по четырём стратегиям.
- Параметры стратегий зашейте в `spec.md`: доходность, волатильность, ликвидность, налог.
- Для каждой стратегии – отдельный график капитала + риск-метрика (например, *max drawdown*).
- Финал – рекомендация: «при ваших параметрах подходит стратегия X, но учтите Y».
- *Verification* обязательна: на тестовых параметрах должны получаться ожидаемые порядки величин.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

– сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ есть `spec.md` в корне с явно перечисленными критериями готовности;
- ✓ работа велась по фазам (видно в логе *TodoWrite*);
- ✓ четыре стратегии видимо отличаются по графику и по метрикам;
- ✓ есть *verification*-блок, в котором Claude явно проверил числа на тестовом сценарии;
- ✓ финальная рекомендация «что подходит вам» зависит от параметров формы (попробуйте поменять – рекомендация должна меняться);
- ✓ Playwright скриншот UX зафиксирован (например, в `screenshots/`).

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Прогони красную команду через мой симулятор: представь, что ты скептик-аналитик. Найди три места, где модель «лжёт» или сильно упрощает реальность. Что я должен помнить, показывая это другу?»

Глубокий анализ одной категории расходов

Цель: вариация проекта А – те же выписки, но только одна категория (например, «еда вне дома», «транспорт» или «развлечения»). Глубокий разбор: где, когда, у кого, в какие дни недели.

Подсказки

- Полный цикл spec → план → фазы → verification.
- В spec явно укажите выбранную категорию и три-четыре «среза» (по дням недели, по поставщикам, по времени суток, по размеру операции).
- Финал: краткий `category-report.md` с тремя «открытиями» и одной рекомендацией.
- Используйте Playwright для ревизии UX отчёта.

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

— сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ в `spec.md` явно указаны: выбранная категория, источник данных, четыре среза, критерии готовности;
- ✓ есть `category-report.md` со структурированным анализом и тремя открытиями;
- ✓ работа велась фазами – видно по TodoWrite-логу или коммитам;
- ✓ числа сверены: общая сумма по категории совпадает с `data/all-ops.csv` (или вашими исходными выписками);
- ✓ отчёт «не даёт советов из воздуха» – каждая рекомендация подкреплена конкретным наблюдением из данных.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Если бы тебе показали этот отчёт без знания обо мне, какие пять вопросов ты бы задал автору категории, чтобы понять контекст? Какие из этих вопросов я мог бы зашить в следующий отчёт сразу?»

Что добавилось к словарю

Plan Mode	Режим Claude Code, в котором он <i>гарантированно</i> не трогает ваши файлы, пока вы не утвердите план. Включается через Shift+Tab. Незаменим для крупных задач.
TodoWrite	Встроенный инструмент Claude Code для ведения списка подзадач прямо в диалоге. Каждая задача имеет статус: pending, in_progress, completed. Видно, где Claude и что забыто.
Спецификация (spec.md)	Документ-ТЗ для крупной задачи. Содержит: цель, входы, выходы, шаги, критерии готовности, что вне scope, риски и вопросы. Claude работает по нему как стажёр по брифу.
Compaction	Команда /compact: «уплотнить сессию». Claude саммаризирует диалог и удаляет промежуточные сообщения. Используется между фазами крупной задачи и в конце дня.
/clear	Полная очистка контекста. Используется реже – когда хотите начать «с нуля» в той же папке, без истории.
Verification	Привычка проверять результат прежде чем сказать «готово». Запустить, прогнать тестовый сценарий, сверить с критериями, сделать Playwright-скриншот. Skill: superpowers:verification-before-completion.
Декомпозиция	Разбиение крупной задачи на фазы – по данным, по артефактам или по ревью. Каждая фаза самостоятельно полезна, измерима, укладывается в 1–3 часа.
Фаза	Один логический блок крупной задачи. Например: «парсинг», «категоризация», «отчёт». Между фазами – ревью и /compact.
Критерии готовности	Список измеримых условий, по которым задача считается готовой. Например: «график показывает все 12 месяцев», «total spend сходится с суммой по категориям». Записываются в spec.md.
Out of scope	Раздел спес, в котором перечислено, что мы <i>не будем</i> делать в этой задаче. Важно, чтобы Claude не уплывал за границы.
Sanity-check	Быстрая проверка результата на «разумность»: сходятся ли суммы, есть ли все ожидаемые поля, не показывает ли система абсурдных значений. Часть verification.

Семь вопросов на закрепление

01 Какие признаки говорят, что задача слишком большая для одного промпта?

Больше 10 шагов; задача завязана на несколько источников данных; есть несколько независимых выходов (отчёт + дашборд + автоматизация); вы сами не уверены, что Claude должен сделать. В этих случаях – Plan Mode, спес, фазы.

02 Чем Plan Mode отличается от обычного режима с точки зрения файлов?

В Plan Mode Claude *гарантированно* не изменяет ни одного файла. Только читает существующие, изучает структуру, составляет план. Файлы трогаются только после вашего «утверждено» и выхода в default.

03 Что обязательно должно быть в `спес.md`?

Семь обязательных разделов: **цель, входы, выходы, шаги (фазы), критерии готовности, что вне score, риски и вопросы**. Без каждого из них Claude может «уплыть» по-своему.

04 Что делает `/compact` и чем отличается от `/clear`?

`/compact` саммаризирует диалог: сохраняет цель, ключевые решения и текущее состояние, удаляет промежуточные сообщения. Контекст «лёгкий», но Claude помнит суть. `/clear` – полная очистка, контекст теряется. Compact используется чаще – между фазами; clear – редко, для «старта с нуля».

05 Зачем нужно **verification-before-completion**?

Самая частая ошибка крупных задач – «соврать про готовность»: код выглядит правильным, объявляется «готово», но при запуске не работает. Verification требует прежде чем сказать «готово»: запустить, прогнать тестовый сценарий, сверить с критериями spec, сделать Playwright-скриншот. Только потом – «done».

06 Как разбивать крупную задачу на фазы — назовите три способа.

По данным (парсинг → нормализация → агрегация → визуализация); **по артефактам** (сначала data/operations.csv, потом dashboard.html, потом report.md); **по ревью** (каждая фаза заканчивается `/compact` и вашим утверждением).

07 Какой простой **sanity-check** вы бы прогнали для «анализа выписок за год»?

Минимум: число строк в `data/all-ops.csv` равно сумме строк во всех 12 исходных файлах. Total spend в отчёте равна сумме всех «расход»-строк. Сумма по категориям равна total. Все 12 месяцев присутствуют в графике тренда. Если хоть один пункт «X» – «готово» нельзя говорить.

Что я теперь умею

- ☐ Уверенно переключаюсь в Plan Mode через `Shift+Tab` и понимаю, когда он нужен.
- ☐ Пишу `spec.md` с семью разделами для крупной задачи.
- ☐ Использую TodoWrite, чтобы видеть прогресс по шагам.
- ☐ Применяю `/compact` между фазами и в конце дня.
- ☐ Делаю verification-before-completion: тестовый сценарий, сверка с критериями, Playwright-скриншот.
- ☐ Декомпозирую крупные задачи на фазы (по данным, артефактам или ревью).
- ☐ Сделал четыре проекта: анализ выписок, прогноз FI, симулятор пенсии, глубокий анализ категории.

Готовы закрыть этап? Нажатие фиксирует прогресс на главной странице.

«План — не замедление работы. План — страховка от того, чтобы делать неправильное быстро».

Этап 04 пройден — крупные задачи без потери цели стали возможны. Далее учим Claude публиковать вашу работу в интернет — через git, GitHub и worktrees.

ВНУТРИ ЭТАПА

- § 0 · Подготовка
- § I · Темы
- § III · Проекты
- § V · Словарь
- § VI · Quiz
- § VII · Чек-лист

КУРС

- К оглавлению
- ← Этап 03
- Этап 05